Advanced Dynare

Chapter II

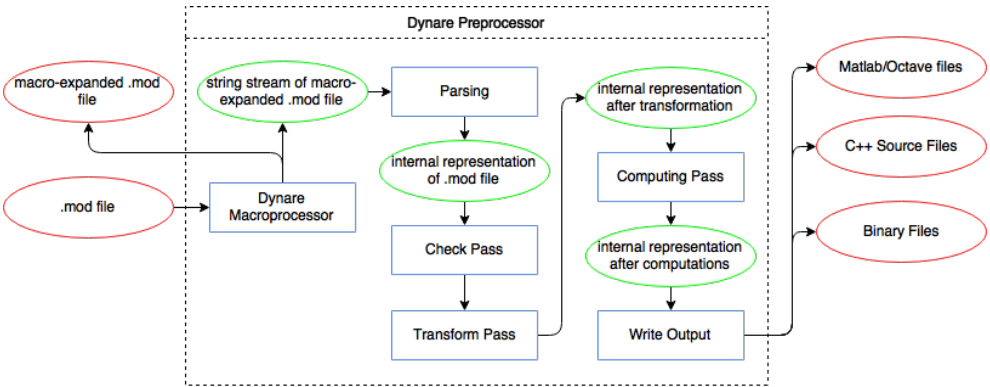
The Dynare preprocessor

Johannes Pfeifer

University of the Bundeswehr Munich

October 2025, Bank of Korea

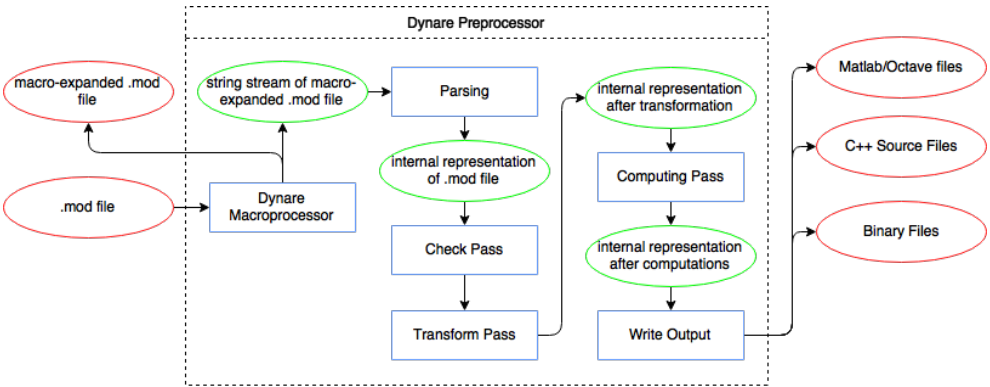
Overview


Dynare Preprocessor
macro-expanded .mod file
string stream of macro-expanded .mod file
Parsing
internal representation after transformation
Matlab/Octave files
C++ Source Files
Binary Files
internal representation of .mod file
Computing Pass
.mod file
Dynare Macroprocessor
Check Pass
internal representation after computations
Transform Pass
Write Output

Outline

Outline

Overview


Dynare Preprocessor
macro-expanded .mod file
string stream of macro-expanded .mod file
Parsing
internal representation after transformation
Matlab/Octave files
C++ Source Files
Binary Files
internal representation of .mod file
Computing Pass
.mod file
Dynare Macroprocessor
Check Pass
internal representation after computations
Transform Pass
Write Output

Calling Dynare

- Dynare is called from the host language platform with the syntax

```
dynare FILENAME[.mod]
```

- This call can be followed by certain options:
  - Some of these options impact host language platform functionality
    → *e.g.* `nograph` prevents graphs from being displayed in MATLAB
  - Some cause differences in the output created by default
    → `notmpterms` prevents temporary terms from being written to the static/dynamic files
  - Others impact the functionality of the macroprocessor or the preprocessor
    → *e.g.* `nostrict` shuts off certain checks that the preprocessor does by default

```
dynare FILENAME[.mod] nostrict
```

- Matlab command line supports syntax completion since Dynare 6.x

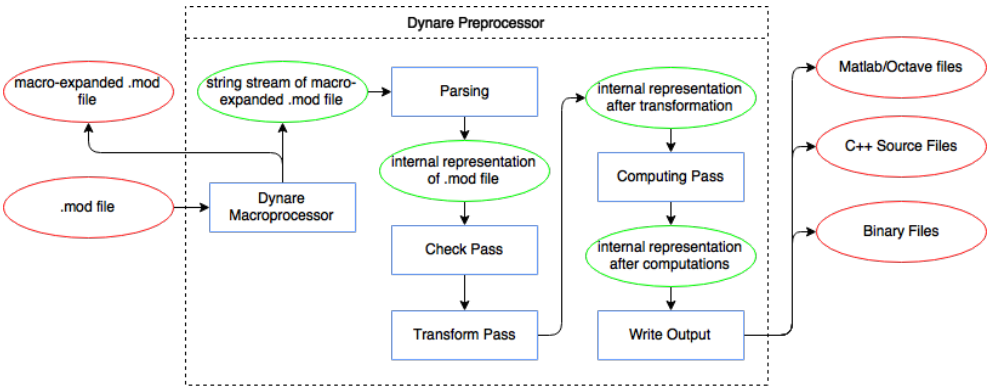
Command line options in the mod-file

- Command line options can alternatively be defined in the first line of the .mod file
- Avoids to always have to invoke an option at the command line
  → particularly useful for the `nostrict` option during the development phase of a model
- Definition must be
  - a one-line Dynare comment, i.e. begin //
  - the options must be enclosed in `--+ options:` and `+--` and must be whitespace separated
  - As in the command line, if an option admits a value, the equal symbol must not be surrounded by spaces

```
// --+ options: json=compute, stochastic, nostrict +--
```

Outline

Overview


Dynare Preprocessor
macro-expanded .mod file
string stream of macro-expanded .mod file
Parsing
internal representation after transformation
Matlab/Octave files
C++ Source Files
Binary Files
internal representation of .mod file
Computing Pass
.mod file
Dynare Macroprocessor
Check Pass
internal representation after computations
Transform Pass
Write Output

Macro processing

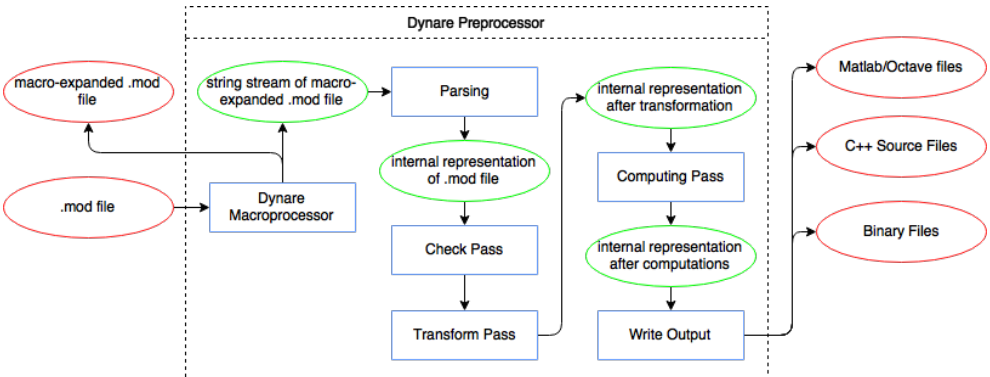
- The Dynare macro language provides a set of macro commands that can be used in `mod` files
- The macro processor employs text expansions/inclusions to transform a `mod` file with macro commands into a `mod` file without macro commands
  → result can be stored using `savemacro` option
- The result is fed to the parser
  → use `onlymacro` to stop after macro processing before parsing

```
dynare FILENAME[.mod] onlymacro savemacro=macroresult
```

- Attention: the macro processor only does text substitution; objects computed in later steps are not available yet and cannot be conditioned on for that reason

Outline

Overview


Dynare Preprocessor
macro-expanded .mod file
string stream of macro-expanded .mod file
Parsing
internal representation after transformation
Matlab/Octave files
C++ Source Files
Binary Files
internal representation of .mod file
Computing Pass
.mod file
Dynare Macroprocessor
Check Pass
internal representation after computations
Transform Pass
Write Output

Parsing overview

- Parsing is the action of transforming an input text (a `mod` file in our case) into a data structure suitable for computation
- The parser consists of three components:
  1. the lexical analyzer, which recognizes the "words" of the `mod` file (analog to the *vocabulary* of a language)
  2. the syntax analyzer, which recognizes the "sentences" of the `mod` file (analog to the *grammar* of a language)
  3. the parsing driver, which coordinates the whole process and constructs the data structure using the results of the lexical and syntax analyses

Undesired Parsing

- Dynare will try to parse all tokens it recognizes
- Code not recognized by the parser is directly passed to the preprocessor output
  $\rightarrow$ allows using MATLAB/Octave commands directly in the mod-file
- Causes problems when one wants to invoke commands containing recognized tokens

Bypassing the parsing

- The `verbatim;` block instructs Dynare not to parse text contained in it
- In the following example, `stoch_simul` would otherwise be recognized as a Dynare command during parsing

```
verbatim;
rhos = [ 0.8, 0.9, 1];
for iter = 1:length(rhos)
    set_param_value('rho',rhos(iter));
    [info, oo_, options_, M_] = stoch_simul(M_, options_, oo_, var_list_)
    if info(1)~=0
        error('Simulation failed for parameter draw')
    end
end
end;
```

1. Lexical analysis

- The lexical analyzer recognizes the "words" (or lexemes) of the language
- When invoked, the lexical analyzer reads the next characters of the input, tries to recognize a lexeme, and either produces an error or returns the associated token

2. Syntax analysis

- The `mod` file grammar is transformed into C++ source code by the program `bison`
- The grammar tells a story which looks like:
  - A `mod` file is a list of statements
  - A statement can be a `var` statement, a `varexo` statement, a `model` block, an `initval` block, ...
  - A `var` statement begins with the token `VAR`, then a list of `NAME`s, then a semicolon

Syntax analysis

- Using the list of tokens produced by lexical analysis, the syntax analyzer determines which "sentences" are valid in the language, according to a grammar composed of rules.
- `(1+3)*2; 4+5;` will pass the syntax analysis without error
- `1++2;` will fail the syntax analysis, even though it has passed the lexical analysis

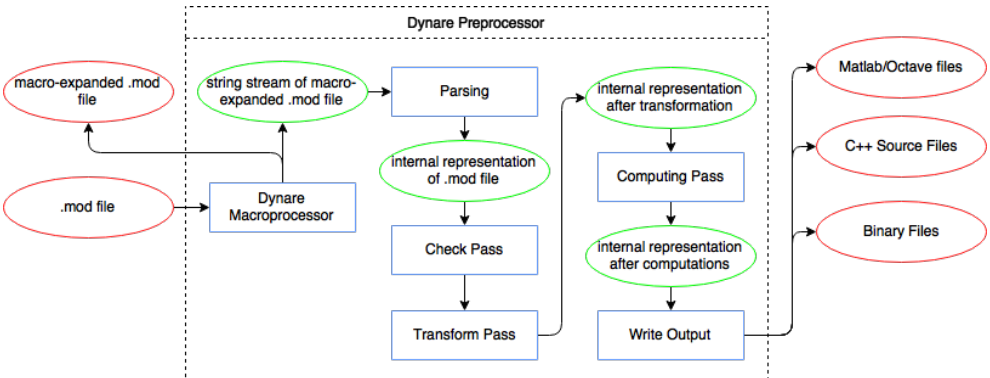
3. Parsing driver

- The parsing driver opens the `mod` file and launches the lexical and syntactic analyzers
- It implements most of the semantic actions of the grammar
  → creates data structure representing the `mod` file
- If there is a parsing error
  - unknown keyword
  - syntax errors
  - use of undeclared symbols in model block, `initval` block...
  - use of a symbol incompatible with its type (*e.g.* parameter in `initval`, local variable used both in model and outside model)
  - ...

  it displays the line and column numbers where the first error occurred and exits
  → there may be others only displayed when the first error is fixed

Outline

Overview

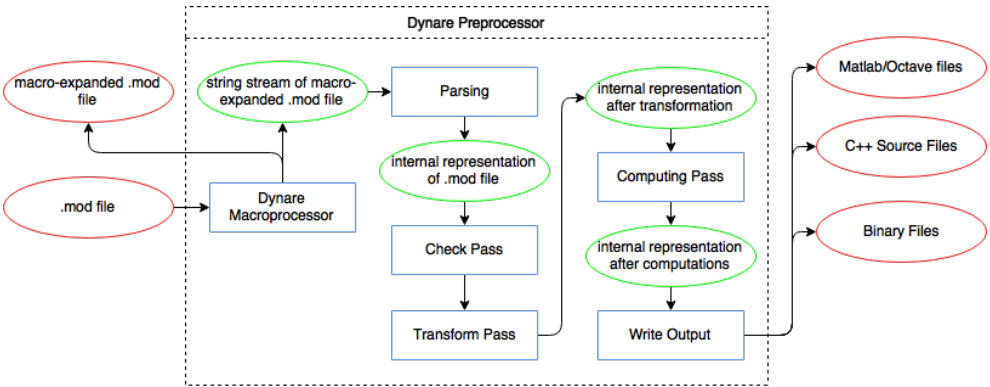

Dynare Preprocessor
macro-expanded .mod file
string stream of macro-expanded .mod file
Parsing
internal representation after transformation
Matlab/Octave files
C++ Source Files
Binary Files
internal representation of .mod file
Computing Pass
.mod file
Dynare Macroprocessor
Check Pass
internal representation after computations
Transform Pass
Write Output

Check pass

- Some errors in the `mod` file can be detected during parsing, but some other checks can only be done when parsing is completed...
- The check pass performs many checks. Examples include:
  - check there is at least one equation in the model (except if doing a standalone BVAR estimation)
  - checks for coherence in statements (*e.g.* options passed to statements do not conflict with each other, required options have been passed)
  - checks for coherence among statements (*e.g.* if `osr` statement is present, ensure `osr_params` and `optim_weights` or `planner_objective` statements are present (and not both))
  - checks for coherence between statements and attributes of `mod` file (*e.g.* `use_dll` is not used with `block` or `bytecode`)

Outline

Overview


Dynare Preprocessor
macro-expanded .mod file
string stream of macro-expanded .mod file
Parsing
internal representation after transformation
Matlab/Octave files
C++ Source Files
Binary Files
internal representation of .mod file
Computing Pass
.mod file
Dynare Macroprocessor
Check Pass
internal representation after computations
Transform Pass
Write Output

Transform pass (1/2)

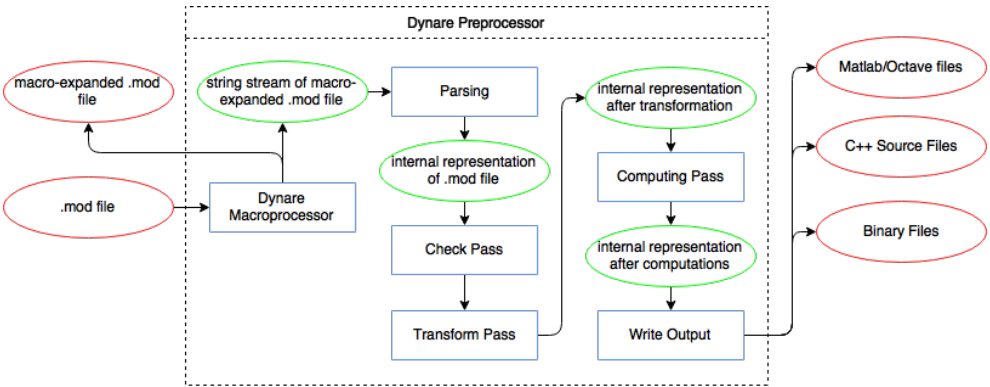
- The transform pass makes necessary transformations to prepare the `ModFile` object for the computing pass. Examples of transformations include:
  - creation of auxiliary variables and equations for leads, lags, expectation operator, differentiated forward variables, etc.
    → more information stored in `M_.aux_vars` and at Wiki
  - detrending of model equations if nonstationary variables are present
  - decreasing leads/lags of `predetermined_variables` by one period
  - addition of FOCs of Lagrangian for `ramsey_model`
  - setup of OccBin structure
  - addition of `dsge_prior_weight` initialization before all other statements if estimating a DSGE-VAR where the weight of the DSGE prior of the VAR is calibrated

Transform pass (2/2)

- Finally, checks are performed on the transformed model
- Examples include:
  - same number of endogenous variables as equations (not done in certain situations, *e.g.* `ramsey_model`, `discretionary policy`, where checks are done in Matlab)
  - correspondence among variables and statements, *e.g.* Ramsey, identification, perfect foresight solver, and perfect foresight solver are incompatible with `var_exo_det`
  - correspondence among statements, *e.g.* for DSGE-VAR with `bayesian_irf` option, the number of shocks must be equal to the number of observed variables

Outline

Overview

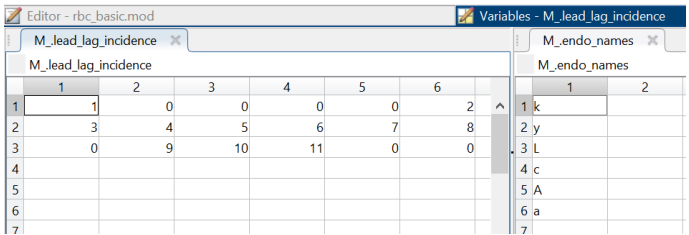

Dynare Preprocessor
macro-expanded .mod file
string stream of macro-expanded .mod file
Parsing
internal representation after transformation
Matlab/Octave files
C++ Source Files
Binary Files
internal representation of .mod file
Computing Pass
.mod file
Dynare Macroprocessor
Check Pass
internal representation after computations
Transform Pass
Write Output

Overview of the computing pass

- Computing pass creates static model from dynamic one (by removing leads/lags)
- Determines which derivatives to compute and computes:
  - lead/lag variable incidence matrix
  - symbolic derivatives w.r.t. endogenous, exogenous, and parameters, if needed
  - equation normalization + block decomposition
  - temporary terms
- Asserts that equations declared linear are indeed linear (by checking that Hessian $== 0$)

Variables in the dynamic model

- `Dynamic Model` is characterized by leag/lag incidence matrix stored in `M_.lead_lag_incidence`:
  - `max_lag + max_lead + 1` rows (usually 3): the first row indicates $t-1$ (if applicable), the second row t, and the third row $t+1$ (if applicable)
  - one column for every endogenous symbol in order of declaration
    $\rightarrow$ includes endogenous and exogenous auxiliary variables created during the transform pass
  - elements of the matrix are either 0 (if the variable does not appear in the model) or correspond to the variable's column in the Jacobian of the dynamic model



Static versus dynamic model

- The static model is simply the dynamic model without leads and lags, used to characterize the steady state
- The Jacobian of the static model is used in the (MATLAB) solver for determining the steady state

Example

- suppose dynamic model is $2x_t \cdot x_{t-1} = 0$
- static model is $2x^2 = 0$, whose derivative w.r.t. x is $4x$
- dynamic derivative w.r.t. x_t is $2x_{t-1}$, and w.r.t. x_{t-1} is $2x_t$
- removing leads/lags from dynamic derivatives and summing over the two partial derivatives w.r.t. x_t and x_{t-1} gives $4x$

Which derivatives to compute?

- In deterministic mode:
  - static Jacobian w.r.t. endogenous variables only
  - dynamic Jacobian w.r.t. endogenous variables only
- In stochastic mode:
  - static Jacobian w.r.t. endogenous variables only
  - dynamic Jacobian w.r.t. endogenous, exogenous, and deterministic exogenous variables
  - dynamic Hessian w.r.t. endogenous, exogenous, and deterministic exogenous variables
  - possibly dynamic 3rd derivatives (if `order` option ≥ 3)
  - possibly dynamic Jacobian and/or Hessian w.r.t. parameters (if `identification` or `analytic_` needed for `estimation` and `params_derivs_order` > 0)
- For Ramsey policy: the same as above, but with one further order of derivation than declared by the user with `order` option

Temporary terms (1/2)

- When the preprocessor writes equations and derivatives in its outputs, it takes advantage of sub-expression sharing
- In MATLAB static and dynamic output files, equations are preceded by a list of **temporary terms**
- These terms are variables containing expressions shared by several equations or derivatives
  → greatly enhances the computing speed of the model residual, Jacobian, Hessian, or third derivative

Original:

```
residual(0)=x+y^2-z^3;
residual(1)=3*(x+y^2)+1;
```

Optimized:

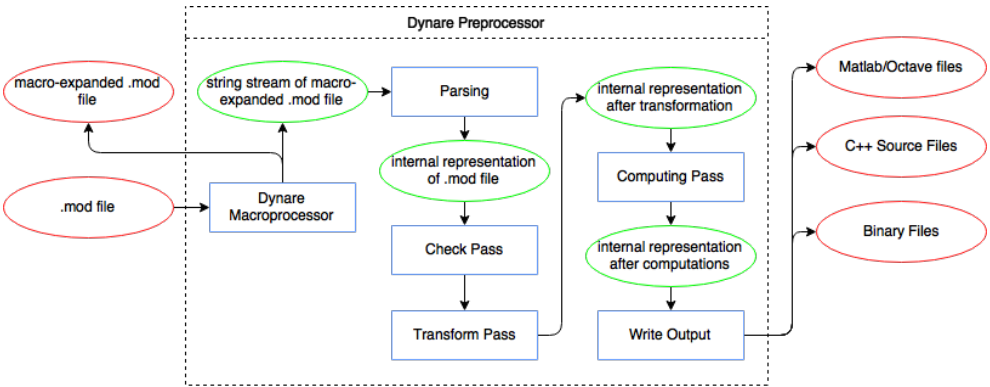
```
T1=x+y^2;
residual(0)=T1-z^3;
residual(1)=3*T1+1;
```

Temporary terms (2/2)

- Expression storage in the preprocessor implements maximal sharing, but this is not optimal for the MATLAB output files, because creating a temporary variable also has a cost (in terms of CPU and of memory)
- Computation of temporary terms implements a trade-off between:
  - cost of duplicating sub-expressions
  - cost of creating new variables
- Algorithm uses recursive cost calculation that marks some nodes as being "temporary"
- *Problem*: redundant with optimizations done by the C/C++ compiler (when Dynare is in DLL mode)
  $\Rightarrow$ compilation very slow on big models

Outline

Overview


Dynare Preprocessor
macro-expanded .mod file
string stream of macro-expanded .mod file
Parsing
internal representation after transformation
Matlab/Octave files
C++ Source Files
Binary Files
internal representation of .mod file
Computing Pass
.mod file
Dynare Macroprocessor
Check Pass
internal representation after computations
Transform Pass
Write Output

Output overview

- If `mod` file is `mymodel.mod`, all created files will be in a namespace `mymodel` (`+mymodel` subfolder)
- Main output file is `mymodel.driver`, containing:
  - general initialization commands
  - symbol table output (variable and parameters names etc.)
  - lead/lag incidence matrix
  - call to MATLAB functions corresponding to the statements of the `mod` file
- Subsidiary output files:
  - for the static model (residuals, temporary terms, derivatives)
  - for the dynamic model (residuals, temporary terms, derivatives)
  - one for the auxiliary variables in the dynamic model (if relevant)
  - one for the steady state file (if relevant)
  - for the planner objective and Lagrange multipliers (static residuals and derivatives, if relevant)
  - one each for the static and dynamic parameter derivatives (if required)